

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение

Дисциплина «Методы оптимизации»

Отчёт по лабораторной работе №7

Вариант 19

Выполнили
Линейский Аким Евгеньевич
Поток 2.3 (Р3215)
Преподаватель
Кудашов Вячеслав Николаевич

Санкт-Петербург, 2026

1 Задание 1. Аналитическое исследование функции

1.1. Область определения функции и вычисление градиента

Дана целевая функция двух переменных по варианту:

$$z(x, y) = 10^{-2}(8x^2 + 5xy + 57x + 30y + 54) \quad (1)$$

Исследование проводится в замкнутой прямоугольной области:

$$(x, y) \in [-20.0, 20.0] \times [-50.0, 50.0] \quad (2)$$

Для поиска стационарных точек вычислим частные производные первого порядка (компоненты вектора градиента ∇z):

$$z'_x = \frac{\partial z}{\partial x} = 10^{-2}(16x + 5y + 57) \quad (3)$$

$$z'_y = \frac{\partial z}{\partial y} = 10^{-2}(5x + 30) \quad (4)$$

1.2. Определение и классификация стационарных точек

Составим систему необходимых условий экстремума первого порядка, приравняв частные производные к нулю:

$$\begin{cases} 16x + 5y + 57 = 0 \\ 5x + 30 = 0 \end{cases} \quad (5)$$

Из второго уравнения системы находим стационарную координату x_0 :

$$5x = -30 \implies x_0 = -6 \quad (6)$$

Подставим полученное значение $x_0 = -6$ в первое уравнение для нахождения координаты y_0 :

$$16(-6) + 5y + 57 = 0 \implies -96 + 5y + 57 = 0 \implies 5y = 39 \implies y_0 = 7.8 \quad (7)$$

Функция имеет единственную стационарную точку внутри области: $P_0(-6.0, 7.8)$.

Для определения её типа найдем матрицу Гессе (частные производные второго порядка):

$$\frac{\partial^2 z}{\partial x^2} = 0.16, \quad \frac{\partial^2 z}{\partial x \partial y} = 0.05, \quad \frac{\partial^2 z}{\partial y^2} = 0 \quad (8)$$

Составим определитель матрицы Гессе (дискриминант квадратичной формы Δ):

$$\Delta = \det(H) = \begin{vmatrix} 0.16 & 0.05 \\ 0.05 & 0 \end{vmatrix} = 0.16 \cdot 0 - (0.05)^2 = -0.0025 \quad (9)$$

Так как определитель матрицы Гессе отрицателен ($\Delta < 0$), стационарная точка $P_0(-6.0, 7.8)$ является **седловой точкой**. Локальных экстремумов внутри области нет.

1.3. Нахождение глобального минимума

Поскольку внутри области локальные экстремумы отсутствуют, а сама функция линейна по переменной y , глобальные экстремумы могут достигаться исключительно на границе прямоугольной области, а именно — в её угловых точках. Вычислим значения функции в вершинах прямоугольника:

- $z(-20, -50) = 56.14$
- $z(-20, 50) = -13.86$
- $z(20, 50) = 109.94$
- $z(20, -50) = -21.06$

Ответ: Точка глобального минимума равна $X_{min} = (20, -50)$, минимальное значение функции на заданной области составляет $z(X_{min}) = -21.06$.

1.4. Программный модуль отрисовки линий уровня

Ниже представлен Python-скрипт, вычисляющий матрицу уровней функции и строящий изолинии рельефа с указанием положения седловой точки.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def objective(x, y):
5     return 1e-2 * (8 * x**2 + 5 * x * y + 57 * x + 30 * y + 54)
6
7 # Сетка для построения
8 x = np.linspace(-20.0, 20.0, 400)
9 y = np.linspace(-50.0, 50.0, 400)
10 X, Y = np.meshgrid(x, y)
11 Z = objective(X, Y)
12
13 plt.figure(figsize=(8, 6))
14 contours = plt.contour(X, Y, Z, levels=25, cmap='viridis')
15 plt.clabel(contours, inline=True, fontsize=8)
16 plt.plot(-6.0, 7.8, 'ro', markersize=8, label='Седловая точка (-6; 7.8)')
17
18 plt.title('Линии уровня целевой функции')
19 plt.xlabel('x')
20 plt.ylabel('y')
21 plt.grid(True, linestyle='--', alpha=0.6)
22 plt.legend()
23 plt.tight_layout()
24 plt.savefig('contour_plot.png', dpi=300)
25 plt.show()
```

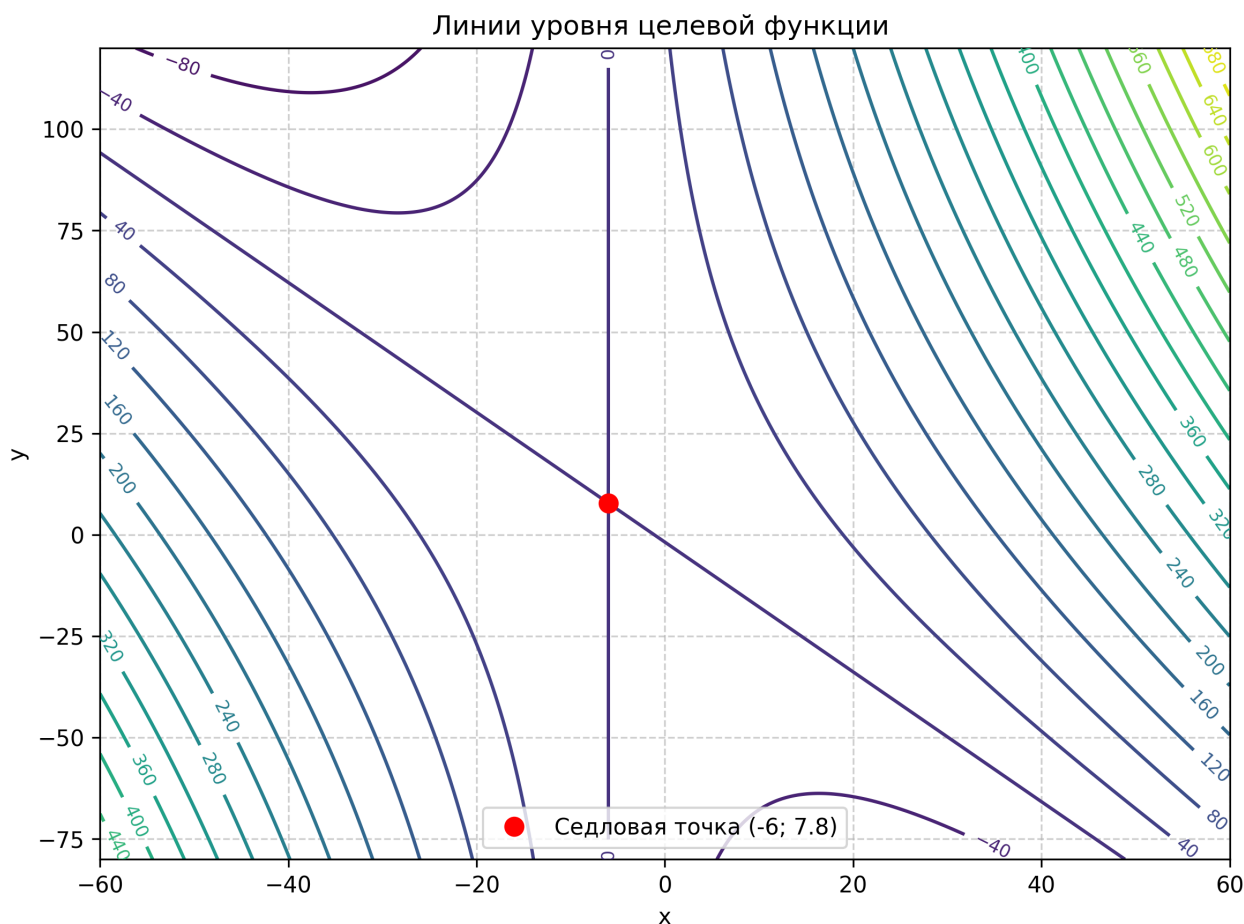


Рис. 1: Линии уровня целевой функции и положение седловой точки

2 Задание 2. Программное моделирование методов оптимизации

2.1. Реализация SGD и застревание в седловой точке

Классический градиентный спуск со скоростью обучения $\alpha = 0.3$ и начальной точки $(-15.0, 40.0)$ скатывается на дно оврага и полностью прекращает движение по оси y на 98-й итерации из-за падения градиента.

2.2. Преодоление седла с помощью реализованного RMSProp

Ниже приведена совмещенная программная реализация методов SGD и адаптивного метода RMSProp (с возможностью переключения параметра памяти γ), включающая функции логирования и отрисовки сравнительных траекторий.

```

1 def gradient(x, y):
2     return np.array([1e-2 * (16 * x + 5 * y + 57), 1e-2 * (5 * x + 30)
3                       ])
4
5 def run_sgd(start_pt, lr=0.3, steps=98):
6     history = [start_pt.copy()]
7     curr = start_pt.copy()
8     for _ in range(steps):

```

```

8         curr -= lr * gradient(curr[0], curr[1])
9         history.append(curr.copy())
10    return np.array(history)
11
12 def run_rmsprop(start_pt, lr=0.3, gamma=0.99, steps=98):
13     history = [start_pt.copy()]
14     curr = start_pt.copy()
15     G = np.zeros(2)
16     eps = 1e-8
17     for _ in range(steps):
18         grad = gradient(curr[0], curr[1])
19         G = gamma * G + (1 - gamma) * (grad ** 2)
20         curr -= (lr / (np.sqrt(G) + eps)) * grad
21         history.append(curr.copy())
22     return np.array(history)
23
24 # Генерация данных траекторий
25 start = np.array([-15.0, 40.0])
26 path_sgd = run_sgd(start)
27 path_rmsprop_good = run_rmsprop(start, gamma=0.99)
28 path_rmsprop_bad = run_rmsprop(start, gamma=0.1)
29
30 # Отрисовка графиков траекторий
31 plt.figure(figsize=(10, 8))
32 plt.contour(X, Y, Z, levels=30, cmap='coolwarm', alpha=0.4)
33 plt.plot(path_sgd[:, 0], path_sgd[:, 1], 'ro-', label='SGD (Застревает
34 в седле)', markersize=2)
35 plt.plot(path_rmsprop_good[:, 0], path_rmsprop_good[:, 1], 'g^-', label=
36 'RMSPProp ($\gamma=0.99$ - Плавный)', markersize=2)
37 plt.plot(path_rmsprop_bad[:, 0], path_rmsprop_bad[:, 1], 'bs-', label='
38 RMSPProp ($\gamma=0.1$ - Пилообразный)', markersize=2)
39 plt.plot(-6.0, 7.8, 'kX', markersize=10, label='Седло')
40
41 plt.title('Сравнение траекторий оптимизации в области локализации')
42 plt.xlabel('x')
43 plt.ylabel('y')
44 plt.legend()
45 plt.grid(True)
46 plt.savefig('paths_comparison.png', dpi=300)
47 plt.show()

```

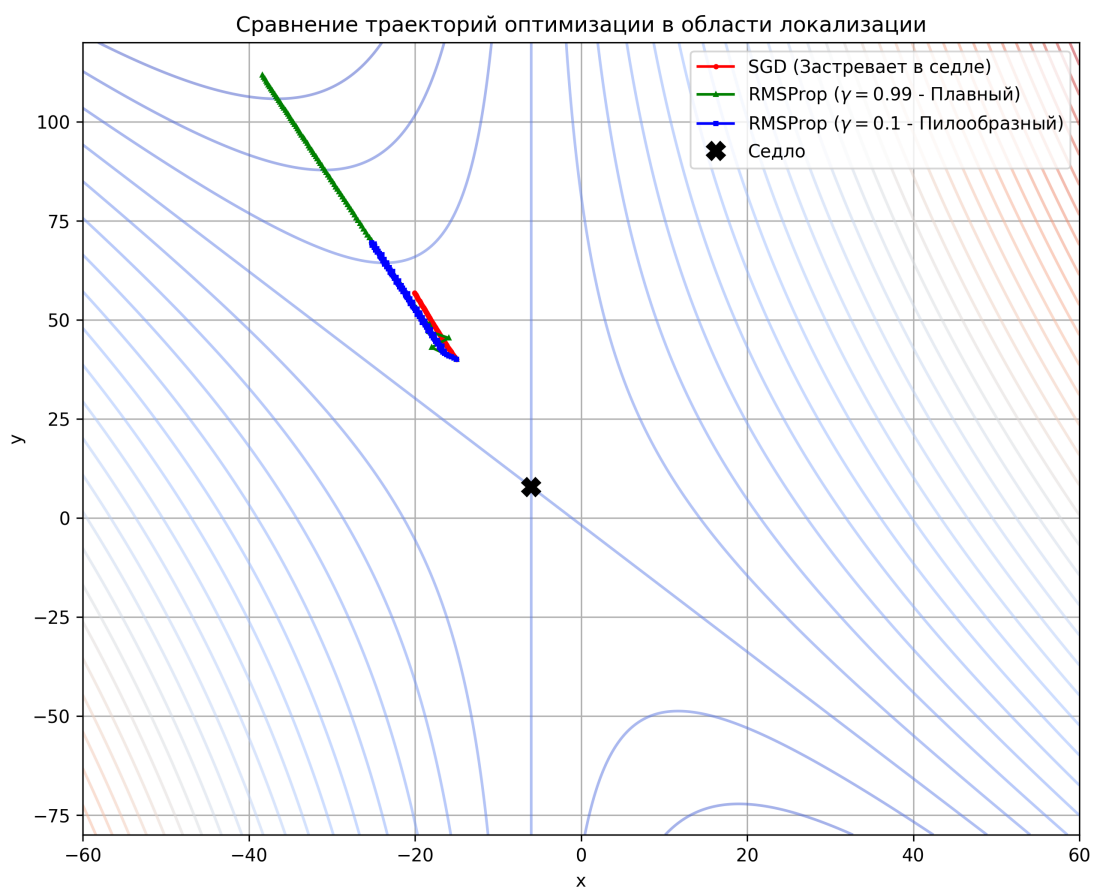


Рис. 2: Траектории классического градиентного спуска и RMSProp при различных значениях γ

3 Задание 3. Реализация эвристики в PyTorch (Adam)

Ниже представлен код скрипта, использующего встроенный в фреймворк PyTorch оптимизатор 'Adam' для автоматического дифференцирования и расчета шагов спуска. Код включает блок визуализации результирующей диаграммы.

```
1 import torch
2
3 solution = torch.tensor([-15.0, 40.0], requires_grad=True)
4 optimizer = torch.optim.Adam([solution], lr=0.3)
5 trace = []
6
7 for i in range(98):
8     optimizer.zero_grad()
9     score = 1e-2 * (8 * solution[0]**2 + 5 * solution[0] * solution[1]
10         + 57 * solution[0] + 30 * solution[1] + 54)
11     score.backward()
12     trace.append(solution.clone().detach().numpy())
13     optimizer.step()
14
15 trace = np.array(trace)
```

```

15
16 # Отрисовка диаграммы последовательных приближений PyTorch Adam
17 plt.figure(figsize=(8, 6))
18 plt.contour(X, Y, Z, levels=25, cmap='plasma', alpha=0.5)
19 plt.plot(trace[:, 0], trace[:, 1], 'm.-', linewidth=1.5, label='Траекто
    рия Adam (PyTorch)')
20 plt.plot(-15.0, 40.0, 'go', label='Старт')
21 plt.plot(20.0, -50.0, 'b*', markersize=10, label='Глобальный минимум')
22 plt.legend()
23 plt.grid(True)
24 plt.title('Последовательные приближения оптимизатора Adam')
25 plt.savefig('pytorch_adam.png', dpi=300)
26 plt.show()

```

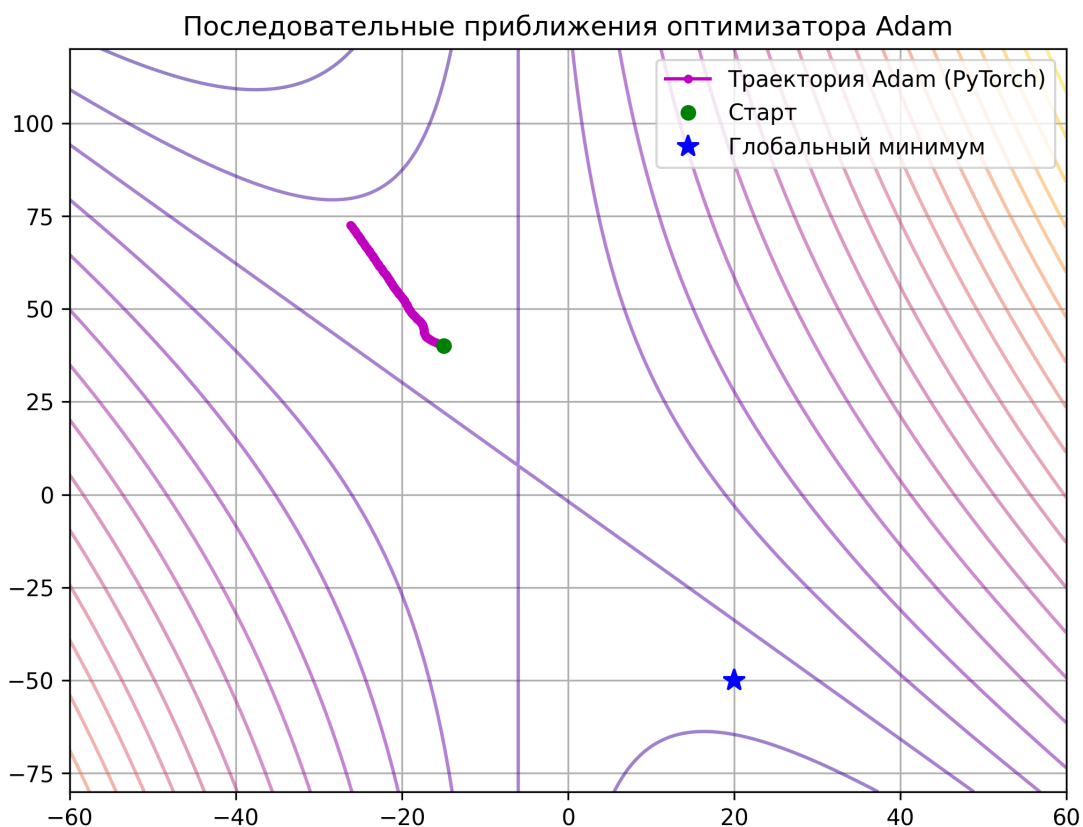


Рис. 3: Диаграмма последовательных приближений оптимизатора Adam в PyTorch